

CSC2103: Data Structures and Algorithms

Group Project

Semester: April 2026

Group Size: 4 – 6 Students

Total Marks: 20

Deadline: 9th August 2026

Submission Method: E-learn

Objective

This group assignment assesses your ability to design, implement, test, and document algorithmic solutions using selected data structures and algorithmic problem-solving strategies.

Each group is required to collaboratively develop **three distinct algorithm-based solutions**, selected from three major algorithmic approaches:

1. Greedy Algorithms
2. Dynamic Programming or Backtracking
3. Heuristic Algorithms

The assignment emphasizes manual implementation, logical problem-solving, teamwork, testing, and responsible use of generative AI tools.

Assignment Overview

Each group must solve **three problems**, with **one problem selected from each category** below.

Problem 1: Greedy Algorithms

Choose **one** from the following:

- Activity Selection Problem
- Fractional Knapsack Problem
- Job Sequencing with Deadlines
- Huffman Coding Algorithm
- Minimum Spanning Tree using Prim's or Kruskal's Algorithm
- Dijkstra's Shortest Path Algorithm

The solution should clearly show how the greedy choice is made at each stage and why the selected approach is suitable for the problem.

Problem 2: Dynamic Programming or Backtracking

Choose **one** from the following (**either from Dynamic Programming or Backtracking**):

Dynamic Programming Options

- 0/1 Knapsack Problem
- Coin Change Problem
- Longest Common Subsequence
- Matrix Chain Multiplication
- Edit Distance Problem

Backtracking Options

- N-Queens Problem
- Sudoku Solver
- Maze Path Finder
- Graph Coloring Problem
- Subset Sum Problem

The solution should demonstrate either:

- decomposition into overlapping subproblems and optimal substructure, for Dynamic Programming; or
- systematic search with constraint checking and backtracking, for Backtracking.

Problem 3: Heuristic Algorithms

Choose **one** from the following:

- Travelling Salesman Problem using Nearest Neighbour Heuristic
- Travelling Salesman Problem using Genetic Algorithm
- Knapsack Problem using Genetic Algorithm
- Simulated Annealing for Optimization
- A* Search Algorithm for Path Finding
- Hill Climbing for Optimization

The solution should explain that the algorithm may not always guarantee the optimal answer, but aims to find a good solution efficiently.

Implementation Guidelines

Each problem must be implemented as a functional console-based program.

The program should:

- accept user input for relevant problem parameters;
- process the input using the selected algorithm;
- display the final output clearly;
- include meaningful output formatting such as tables, lists, paths, selected items, solution boards, or cost summaries.

No GUI-based animation or step-by-step visual simulation is required. All algorithms must be implemented manually in your own code. Built-in or library-provided algorithmic solutions are not allowed. For example, students should not use built-in sorting, graph, optimization, or machine learning libraries to solve the core algorithmic task. General-purpose libraries for input/output formatting may be used, but the main algorithmic logic must be written by the group. Any programming language may be used, but **Python or Java is recommended**.

Scope

To keep the project manageable, each group should implement **one complete solution per category only and they may be separate sub programs**.

For each selected problem, students are expected to focus on:

- correct algorithmic logic;
- clear input and output;
- proper code structure;
- testing with different input cases;
- brief explanation of how the algorithm works.

Generative AI Policy

Generative AI tools such as ChatGPT, GitHub Copilot, Gemini, Claude, or similar tools may be used to assist with learning, debugging, code explanation, test case generation, and documentation support.

However, students must:

- understand and verify all AI-assisted content;
- manually check the correctness of the algorithm;
- declare clearly which parts were assisted by AI;
- avoid submitting copied AI-generated code or explanations without proper understanding.

Overreliance on AI, undeclared AI use, or inability to explain the submitted work may result in mark deductions.

Documentation Requirements

Submit a well-structured report in PDF or MS Word format.

Maximum length: 15 pages, excluding references and appendices.

The report should contain:

1. Cover Page

- Course name
- Assignment title
- Group Name / Number
- Student names and IDs

2. Introduction

- Overview of the three selected problems
- Brief rationale for choosing the algorithms
- Programming language and tools used

3. Problem 1: Greedy Algorithm Implementation

- Problem description
- Brief Explanation of the greedy strategy
- Input and output design
- Key code snippets
- Screenshots of sample runs
- Strengths and limitations

4. Problem 2: Dynamic Programming or Backtracking Implementation

- Problem description
- Explanation of the DP or backtracking approach
- Input and output design
- Key code snippets
- Screenshots of sample runs
- Strengths and limitations

5. Problem 3: Heuristic Algorithm Implementation

- Problem description
- Explanation of the heuristic method
- Input and output design
- Key code snippets

- Screenshots of sample runs
- Strengths and limitations
- 6. **Testing and Validation**
 - Test cases used for each problem
 - Expected output vs actual output
 - Explanation of how correctness was checked
- 7. **Challenges and Solutions**
 - Main technical challenges faced
 - How the group solved or managed them
- 8. **Declaration of AI Usage**
 - AI tools used, if any
 - Parts assisted by AI
 - How the group verified the AI-assisted content
- 9. **Conclusion**
- 10. **References**

The individual reflection or individual contribution explanation should **not** be included in this report, as it will be submitted separately.

Each group must submit a single ZIP file via E-learn containing:

1. Final Report
 - PDF or MS Word format
2. Source Code Folder or GitHub Repository Link
 - A well-organized source code folder must be included in the ZIP file.
 - Alternatively, a link to a maintained GitHub repository may also be provided.
 - If a GitHub repository is used, the link must be included clearly in the report or in a source_link.txt file.
 - The repository must be accessible to the lecturer at the time of submission.
 - The repository should contain all source code files, sample inputs/outputs, and any required instructions to run the programs.
 - Filenames must be meaningful with correct description, for example:
 - problem1_huffman.py
 - problem3_tsp_heuristic.py
 - Code should include appropriate comments, especially for key functions and algorithmic logic.

Marking Scheme

Category	Marks
Documentation and overall report quality	5
Problem 1: Greedy Algorithm Implementation	5
Problem 2: Dynamic Programming or Backtracking Implementation	5
Problem 3: Heuristic Algorithm Implementation	5
Total	20

Penalty Deductions

Issue	Deduction
Use of built-in/library algorithm for the main solution	-1 mark per problem
AI used but not declared	-2 marks
Unverified or copied AI-generated content	Up to -5 marks
High Turnitin similarity above 30% without justification/citation	0 for documentation
Missing source code or non-functional program	Deduction based on severity

Bonus Marks

Up to **+1 bonus mark** may be awarded for:

- creative and clear input/output design;
- reusable or modular code structure;
- strong testing and validation;
- meaningful additional features without unnecessarily increasing project complexity.

Late Submission Penalty

- The submission deadline is **9th August 2026, 11:59 PM**.
- Late submissions will be accepted until **16th August 2026, 11:59 PM**, but a **60% deduction** will be applied to the assessed marks.
- Submissions made after **16th August 2026, 11:59 PM** will receive **zero marks**.